

Estensione Avanzata per il Sim-to-Real Transfer nell'Ambiente Hopper: Un'Analisi Comparativa e Proposta di Implementazione Research-Oriented

1. Introduzione: Il Paradosso del Sim-to-Real nella Locomozione Dinamica

L'apprendimento per rinforzo (Reinforcement Learning, RL) ha catalizzato una rivoluzione nel controllo robotico, permettendo a sistemi complessi di apprendere comportamenti di locomozione sofisticati *ab initio*, senza la necessità di derivare analiticamente le equazioni del moto. Tuttavia, il passaggio dalla simulazione alla realtà fisica (Sim-to-Real) rimane il "tallone d'Achille" di questa disciplina. Le policy addestrate in ambienti ideali, come MuJoCo, tendono a fallire catastroficamente quando dispiestate su hardware reale a causa del cosiddetto *Reality Gap*.¹ Questo divario non è meramente una questione di rumore statistico, ma deriva da discrepanze strutturali fondamentali tra il modello fisico simulato e la realtà stocastica e non lineare del robot fisico.³

Nel contesto specifico dell'ambiente CustomHopper-v0, un sistema monopedale sottattuato e intrinsecamente instabile, il margine di errore è minimo. La strategia di base proposta nel corso, ovvero la *Uniform Domain Randomization* (UDR) applicata alle masse dei link¹, rappresenta un punto di partenza necessario ma insufficiente per garantire una robustezza di livello accademico. La UDR, nella sua forma ingenua, soffre di due patologie opposte: se il range di randomizzazione è troppo ristretto, la policy sovrappone (*overfits*) le dinamiche nominali e fallisce in presenza di outlier reali; se il range è troppo ampio, l'agente si trova a dover risolvere istanze fisicamente impossibili o contraddittorie, portando al collasso dell'apprendimento o a policy eccessivamente conservative.⁵

Il presente rapporto di ricerca si pone l'obiettivo di identificare, analizzare e proporre un'estensione "research-inspired" al progetto, che superi i limiti della UDR statica. Attraverso una *Deep Research* sistematica della letteratura recente (2018-2025), sono state isolate tre metodologie candidate che promettono di massimizzare il trasferimento Sim-to-Real agendo su leve distinte del processo di apprendimento: la dimensione temporale (Latenza), la dimensione fisica (Dinamica dei Contatti e Smorzamento) e la dimensione curricolare (Randomizzazione Adattiva).

L'analisi che segue dimostrerà come l'**Automatic Domain Randomization (ADR)** costituisca l'estensione ottimale in termini di rapporto costo-beneficio implementativo e rilevanza

scientifica, offrendo una soluzione elegante al problema della selezione degli iperparametri di randomizzazione e trasformando il processo di training in un problema di *meta-learning* implicito.

2. Analisi Approfondita delle Metodologie Candidate

La selezione delle opzioni si è basata su tre criteri fondamentali:

1. **Rilevanza Accademica:** La tecnica è supportata da pubblicazioni recenti in conferenze top-tier (ICRA, IROS, NeurIPS)?
2. **Fattibilità Implementativa:** È possibile integrarla modificando `custom_hopper.py` e utilizzando le librerie esistenti (Stable Baselines3, MuJoCo bindings) senza riscrivere l'intero physics engine?
3. **Impatto sul Reality Gap:** La tecnica affronta una causa radice del disallineamento Sim-to-Real?

Opzione 1: Iniezione di Latenza Stocastica (Il Gap Temporale)

Fondamenti Teorici

La simulazione fisica standard opera in un regime di *tempo discreto idealizzato*, dove l'osservazione dello stato s_t , il calcolo dell'azione $a_t = \pi(s_t)$, e l'applicazione della stessa avvengono istantaneamente. Nei sistemi robotici reali, questo assunto è violato sistematicamente. Esistono ritardi inevitabili dovuti alla pipeline di percezione, all'inferenza della rete neurale, alla comunicazione bus (es. EtherCAT), e alla risposta elettromeccanica degli attuatori.²

Studi seminali come "*Sim-to-Real Transfer with Stochastic State Transition Delays*"⁷ e lavori recenti sulla pianificazione di copertura² evidenziano come la latenza trasformi il Processo Decisionale di Markov (MDP) in un *Partially Observable MDP* (POMDP). Se il ritardo supera il tempo di passo di controllo (Δt), l'azione corrente a_t influenzerà lo stato solo nei passi futuri s_{t+k} , rompendo la proprietà di Markov $P(s_{t+1}|s_t, a_t)$.² Per un robot instabile come l'Hopper, un ritardo non modellato di 20-30ms è sufficiente per disaccoppiare il ciclo di feedback e causare la caduta immediata.

Valutazione Implementativa

L'implementazione richiede la modifica del metodo `step` in `custom_hopper.py` per introdurre un buffer di azioni.

- **Meccanismo:** Mantenere una coda FIFO di azioni passate. Al passo t , l'ambiente riceve a_t ma esegue a_{t-k} , dove k è campionato stocasticamente da una distribuzione (es. Poisson o Uniforme) ad ogni reset o step.
- **Vantaggi:** Bassa complessità computazionale; non richiede modifiche al motore fisico

MuJoCo; attacca una causa primaria di fallimento nel reale.

- **Criticità:** Richiede spesso l'augmentazione dello spazio di stato (es. impilando le osservazioni o le azioni passate, *Frame Stacking*) per permettere all'agente di "inferire" il ritardo corrente, complicando l'architettura della policy.²

Opzione 2: Randomizzazione della Dinamica dei Giunti e dell'Attrito (Il Gap Fisico)

Fondamenti Teorici

Mentre la randomizzazione della massa (proposta nel progetto base) influisce sull'inerzia, la discrepanza tra simulazione e realtà è spesso dominata dalle forze di contatto e dalla dinamica interna degli attuatori.

1. **Damping (Smorzamento Viscoso):** MuJoCo modella lo smorzamento come lineare ($F_d = -k \dot{q}$), mentre gli attuatori reali presentano attrito di Coulomb, stiction e isteresi non lineari.¹⁰ Errori nella stima del damping portano a policy che oscillano o che non riescono a stabilizzare il robot.
2. **Friction (Attrito di Contatto):** L'Hopper dipende dall'attrito piede-suolo per generare spinta orizzontale. La simulazione tende a sovrastimare l'aderenza. Studi recenti ("*Impact of Static Friction on Sim2Real*"¹²) dimostrano che randomizzare solo la massa è inutile se non si perturba anche il coefficiente di attrito statico e dinamico, poiché la policy impara a fare affidamento su un "grip" infinito irrealistico.

Valutazione Implementativa

Questa opzione sfrutta i *bindings* Python di MuJoCo (mujoco package) per alterare le proprietà `model.dof_damping` e `model.geom_friction` a runtime.

- **Meccanismo:** Nel metodo `reset_model`, si campionano moltiplicatori scalari per il damping di ogni giunto e per l'attrito del geom del piede, modificando le strutture dati C sottostanti prima della simulazione.¹⁴
- **Vantaggi:** Alta fedeltà fisica; affronta direttamente le non-linearità meccaniche; estende logicamente la randomizzazione della massa.
- **Criticità:** È facile generare ambienti "impossibili" (es. attrito zero = ghiaccio, damping infinito = giunto bloccato). Richiede una calibrazione manuale dei range molto delicata per evitare che l'agente apprenda l'impotenza appresa (*learned helplessness*).

Opzione 3: Automatic Domain Randomization (ADR) (Il Gap Curricolare)

Fondamenti Teorici

L'Automatic Domain Randomization (ADR), introdotta da OpenAI per il progetto Dactyl (Rubik's Cube)¹⁶ e raffinata in lavori successivi come Active Domain Randomization⁵, rappresenta lo stato dell'arte nella gestione del curriculum di training.

Il concetto chiave è che la distribuzione dei parametri ambientali $P_{\phi(\xi)}$ non deve essere fissa (come in UDR), ma evolvere in funzione della performance dell'agente. L'ADR definisce un iperspazio di parametri (masse, attriti, smorzamenti) e mantiene dei "confini" (bounds) per ciascuno.

- Se l'agente performa bene ai confini attuali, l'algoritmo "allarga" i confini, esponendo l'agente a variazioni più estreme (aumentando l'entropia del dominio).
- Se l'agente fallisce, i confini si contraggono, focalizzando il training su scenari più semplici.

Questo approccio risolve il dilemma della scelta dei range in UDR: l'agente determina autonomamente la complessità massima che è in grado di gestire, spingendo costantemente la "frontiera" della robustezza senza collassare.¹⁸

Valutazione Implementativa

L'ADR non è una modifica fisica, ma un *algoritmo di controllo* sopra l'ambiente. Richiede di implementare una logica di feedback tra l'algoritmo RL (es. PPO) e l'ambiente.

- **Meccanismo:** Si modifica `custom_hopper.py` per accettare comandi di aggiornamento dei range. Si implementa una Callback in Stable Baselines3 che monitora il reward medio e invoca l'aggiornamento dei parametri dell'ambiente.
- **Vantaggi:** Elimina il *tuning* manuale dei range; garantisce che l'agente sia sempre sfidato al punto giusto (Flow theory); produce policy estremamente robuste; accademicamente molto rilevante.
- **Criticità:** Richiede una strutturazione del codice più complessa (interazione Callback-Ambiente); necessita di definire soglie di performance sensate.

3. Selezione dell'Estensione Ottimale: Automatic Domain Randomization (ADR)

Dopo un'attenta valutazione, si raccomanda l'implementazione dell'**Automatic Domain Randomization (ADR)** applicata non solo alla massa, ma estesa sinergicamente a **Smorzamento dei Giunti (Damping)** e **Attrito (Friction)**.

Giustificazione della Scelta

1. **Massimizzazione del Punteggio (Research-Inspired):** L'ADR è una tecnica algoritmica avanzata che dimostra una comprensione profonda non solo della fisica (randomizzando più parametri), ma anche delle dinamiche di apprendimento (curriculum automatico). È nettamente superiore alla semplice UDR o all'aggiunta di rumore statico.
2. **Robustezza Intrinseca:** A differenza dell'Opzione 2 (randomizzazione fisica pura), l'ADR evita di bloccare l'apprendimento su configurazioni impossibili. Se l'ambiente diventa troppo scivoloso (basso attrito) e l'agente fallisce, l'ADR restringe automaticamente il

range, permettendo all'agente di recuperare.

3. **Fattibilità Tecnica:** Nonostante la sofisticazione teorica, l'implementazione pratica si basa interamente su logica Python di alto livello (gestione di dizionari e soglie) e sull'uso intelligente delle Callbacks di SB3, evitando le complessità di basso livello della modifica del motore fisico o della gestione dei buffer di latenza.
4. **Integrazione con il Codice Esistente:** Il file `custom_hopper.py` possiede già i metodi `sample_parameters` e `set_parameters`¹, che costituiscono l'infrastruttura ideale per innestare la logica ADR.

4. Rapporto Tecnico e Guida all'Implementazione di ADR

Questa sezione descrive in dettaglio come trasformare il CustomHopper in un ambiente ADR-enabled. L'implementazione si articola in tre componenti: la modifica dell'ambiente per supportare range dinamici, l'estensione dei parametri fisici controllati, e la creazione del loop di controllo tramite Callback.

4.1 Teoria dell'Aggiornamento ADR

Definiamo λ come un parametro fisico (es. massa del torso). In ADR, λ viene campionato da una distribuzione uniforme $U(\phi_L, \phi_H)$, dove ϕ_L e ϕ_H sono i confini inferiore e superiore.

L'algoritmo mantiene uno stato interno per ogni parametro: il valore nominale (default) v_{nom} e una "distanza" Δ .

$$\phi_L = v_{nom} - \Delta, \quad \phi_H = v_{nom} + \Delta$$

L'obiettivo è massimizzare Δ (l'entropia) mantenendo la performance dell'agente $J(\pi)$ sopra una soglia t_{high} .

Regola di Aggiornamento:

Ad ogni valutazione (es. ogni 1000 step):

- Se $J(\pi) > t_{high}$: Incrementa $\Delta \leftarrow \Delta + \delta_{step}$ (Espansione).
- Se $J(\pi) < t_{low}$: Decrementa $\Delta \leftarrow \Delta - \delta_{step}$ (Contrazione).

Questo crea un effetto "respiro": l'ambiente diventa più difficile quando l'agente migliora, e più facile se l'agente regredisce, prevenendo il *catastrophic forgetting*.¹⁶

4.2 Estensione dei Parametri Fisici

Per rendere l'ADR efficace nel contesto Sim-to-Real, estenderemo la randomizzazione oltre la massa, includendo:

1. **Torso Mass (m):** Critico per la stabilità inerziale.
2. **Joint Damping (d):** I giunti reali hanno attriti interni variabili. Modificheremo `model.dof_damping`.¹⁰
3. **Ground Friction (μ):** L'aderenza del piede è fondamentale per l'Hopper. Modificheremo `model.geom_friction`.¹²

L'accesso a questi parametri nei nuovi binding di MuJoCo (`import mujoco`) differisce da `mujoco-py`. Le strutture dati come `model.body_mass` sono accessibili come array numpy in lettura/scrittura diretta.¹⁴

4.3 Struttura del Codice

Di seguito è presentato lo schema implementativo per ottenere il massimo dei voti.

Modifica 1: `env/custom_hopper.py`

È necessario modificare la classe per gestire i confini dinamici.

Python

```
import numpy as np
from gymnasium.envs.mujoco import MujocoEnv
import mujoco # Nuovi binding deepmind

class CustomHopper(MujocoEnv, utils.EzPickle):
    def __init__(self, domain=None, **kwargs):
        #... (codice esistente di inizializzazione)...

MujocoEnv.__init__(self, xml_file, frame_skip,...)

# --- ADR INITIALIZATION ---
# Salviamo i valori nominali originali per applicare le variazioni relative
self.original_masses = np.copy(self.model.body_mass)
self.original_damping = np.copy(self.model.dof_damping)
self.original_friction = np.copy(self.model.geom_friction)

# Dizionario di stato ADR.
# 'delta' rappresenta quanto ci allontaniamo dal nominale (in percentuale o assoluto)
self.adr_state = {
    "mass_range": 0.0, # Delta massa (kg o %)
    "damping_range": 0.0, # Delta damping multiplier (es. 0.0 -> range [1.0, 1.0])
}
```

```

        "friction_range": 0.0 # Delta friction multiplier
    }

    # Parametri fissi per ADR
    self.adr_step_size = 0.05 # Quanto aumentare/diminuire il delta
    self.max_mass_delta = 5.0 # Limite fisico sicurezza
    self.min_friction = 0.5 # Evitare attrito 0 (impossibile)

    def sample_parameters(self):
        """
        Campiona i parametri fisici basandosi sui range ADR correnti.
        Viene chiamato all'interno di reset_model().
        """
        params = {}

        # 1. Massa Torso (Indice 1 solitamente è il torso nell'XML Hopper)
        # Campiona un offset uniforme [-delta, +delta]
        m_delta = self.adr_state["mass_range"]
        params["torso_mass"] = self.original_masses + self.np_random.uniform(-m_delta, m_delta)

        # 2. Joint Damping (Moltiplicatore)
        # Campiona un fattore di scala in [1.0 - delta, 1.0 + delta]
        d_delta = self.adr_state["damping_range"]
        # Clamp per evitare damping negativo
        low_d = max(0.1, 1.0 - d_delta)
        high_d = 1.0 + d_delta
        params["damping_scale"] = self.np_random.uniform(low_d, high_d)

        # 3. Friction (Moltiplicatore)
        f_delta = self.adr_state["friction_range"]
        low_f = max(self.min_friction, 1.0 - f_delta)
        high_f = 1.0 + f_delta
        params["friction_scale"] = self.np_random.uniform(low_f, high_f)

    return params

    def set_parameters(self, params):
        """
        Applica i parametri campionati al modello MuJoCo a runtime.
        """
        # Modifica Massa
        self.model.body_mass = params["torso_mass"]

```

```

# Modifica Damping (tutti i DOF)
# model.dof_damping è un array di dimensione (nv,)
self.model.dof_damping[:] = self.original_damping * params["damping_scale"]

# Modifica Friction (tutti i geom)
# model.geom_friction è (ngeom, 3) -> [tangenziale, torsionale, rotolamento]
# Modifichiamo principalmente l'attrito tangenziale (indice 0)
self.model.geom_friction[:, 0] = self.original_friction[:, 0] * params["friction_scale"]

def update_adr(self, performance_metric, threshold_low, threshold_high):
    """
    Metodo chiamato dalla Callback per espandere o contrarre i domini.
    """
    if performance_metric >= threshold_high:
        # Espansione: Rendi il task più difficile aumentando la varianza
        self.adr_state["mass_range"] += self.adr_step_size
        self.adr_state["damping_range"] += self.adr_step_size
        self.adr_state["friction_range"] += self.adr_step_size
        return "expanded"

    elif performance_metric < threshold_low:
        # Contrazione: Semplifica il task
        self.adr_state["mass_range"] = max(0.0, self.adr_state["mass_range"] - self.adr_step_size)
        self.adr_state["damping_range"] = max(0.0, self.adr_state["damping_range"] -
self.adr_step_size)
        self.adr_state["friction_range"] = max(0.0, self.adr_state["friction_range"] -
self.adr_step_size)
        return "contracted"

    return "stable"

def get_adr_info(self):
    """Ritorna lo stato corrente per il logging"""
    return self.adr_state

```

Modifica 2: La Callback di Controllo (adr_callback.py)

Questa classe estende BaseCallback di Stable Baselines3. Monitora la metrica di successo (reward medio degli ultimi episodi) e invia segnali all'ambiente.

Python

```
from stable_baselines3.common.callbacks import BaseCallback
import numpy as np
```

```
class AutomaticDomainRandomizationCallback(BaseCallback):
```

```
    def __init__(self, check_freq: int, verbose: int = 1):
```

```
        super().__init__(verbose)
```

```
        self.check_freq = check_freq
```

```
        # Soglie per l'Hopper (da tarare in base ai risultati base)
```

```
        # Hopper-v4 solve è circa 3000+, ma in training si punta a progressi incrementali
```

```
        self.threshold_high = 2000 # Se reward > 2000, aumenta difficoltà
```

```
        self.threshold_low = 1000 # Se reward < 1000, diminuisci difficoltà
```

```
    def _on_step(self) -> bool:
```

```
        if self.n_calls % self.check_freq == 0:
```

```
            # Recupera il reward medio dal buffer del Monitor
```

```
            # Nota: Assicurarsi che l'ambiente sia wrappato con Monitor
```

```
            ep_info_buffer = self.model.ep_info_buffer
```

```
            if len(ep_info_buffer) > 0:
```

```
                mean_reward = np.mean([ep_info["r"] for ep_info in ep_info_buffer])
```

```
            # Accedi all'ambiente "unwrapped" per chiamare i metodi custom
```

```
            # Gestisce VecEnv accedendo al primo environment
```

```
            env_unwrapped = self.training_env.envs.unwrapped
```

```
            # Esegui update ADR
```

```
            status = env_unwrapped.update_adr(mean_reward, self.threshold_low,
self.threshold_high)
```

```
            # Logging su Tensorboard/Console
```

```
            adr_stats = env_unwrapped.get_adr_info()
```

```
            self.logger.record("adr/mean_reward", mean_reward)
```

```
            self.logger.record("adr/mass_range", adr_stats["mass_range"])
```

```
            self.logger.record("adr/friction_range", adr_stats["friction_range"])
```

```
            if self.verbose > 0 and status != "stable":
```

```
                print(f"Step {self.num_timesteps}: ADR Status {status}. Reward: {mean_reward:.2f}. "
```

```
                    f"New Mass Delta: {adr_stats['mass_range']:.3f}")
```

```
        return True
```

4.4 Risultati Attesi e Discussione

Implementando questa architettura, i risultati sperimentali dovrebbero evidenziare tre fenomeni chiave, fondamentali per la relazione:

1. **Curve di Apprendimento Adattive:** A differenza della UDR, dove la performance può stallare se il task è troppo difficile fin dall'inizio, l'ADR mostrerà una crescita iniziale rapida (quando i range sono vicini a zero, simile all'ambiente deterministico). Successivamente, si osserverà una fase di plateau o leggera decrescita del reward, che però corrisponderà a un aumento delle curve `adr/mass_range` e `adr/friction_range` nei grafici di Tensorboard. Questo indica che l'agente sta mantenendo la competenza mentre il task diventa progressivamente più difficile.⁵
2. **Robustezza Sim-to-Sim:** Testando la policy finale sull'ambiente CustomHopper-target-v0 (che ha una massa diversa e non nota a priori), l'agente addestrato con ADR dovrebbe mostrare una varianza di successo molto minore rispetto a un agente addestrato con massa fissa, e potenzialmente superiore a uno UDR se i range UDR erano mal calibrati (troppo ampi o troppo stretti).¹⁸
3. **Prevenzione dell'Overfitting:** I grafici mostreranno che i confini dell'ADR "respirano". Se l'agente incontra una regione dello spazio dei parametri dove la policy fallisce, il reward crolla, causando una contrazione automatica dei range. Questo permette all'agente di recuperare e riapprendere, una caratteristica di auto-guarigione assente nella UDR statica.²⁰

4.5 Integrazione e Validazione

Per validare il sistema, si consiglia di lanciare un training di almeno 500k timesteps. I log dovranno mostrare la correlazione inversa tra l'aumento della difficoltà (delta dei parametri) e i picchi di reward, stabilizzandosi infine su un "volume di competenza" massimo. Questo dimostra non solo la capacità di risolvere il task, ma di risolverlo sotto la massima incertezza possibile, che è la definizione operativa di una policy pronta per il Sim-to-Real.

5. Conclusione

L'estensione proposta eleva il progetto da un semplice esercizio di tuning di parametri a un'implementazione di algoritmi di Meta-Learning allo stato dell'arte. L'Automatic Domain Randomization affronta il problema centrale del Sim-to-Real non cercando di modellare perfettamente la realtà (che è impossibile), ma preparando l'agente a sopravvivere a una distribuzione di realtà sempre più vasta. L'implementazione sinergica su massa, smorzamento e attrito, gestita da un loop di feedback automatico, rappresenta il gold standard per la ricerca attuale nella locomozione robotica e garantisce la massima valorizzazione del progetto in sede d'esame.

Riferimenti Bibliografici Chiave

- ⁷
Sandha et al., "*Sim2Real Transfer for Deep Reinforcement Learning with Stochastic State Transition Delays*", 2021. (Per la teoria sulla latenza e i POMDP).
- ¹²
Gang et al., "*Impact of Static Friction on Sim2Real in Robotic Reinforcement Learning*", 2025. (Per l'importanza critica di attrito e damping).
- ⁵
Mehta et al., "*Active Domain Randomization*", PMLR 2020. (Il paper di riferimento per l'algoritmo ADR proposto).
- ¹⁶
OpenAI et al., "*Solving Rubik's Cube with a Robot Hand*", 2019. (Applicazione fondamentale dell'ADR).
- ¹⁴
DeepMind, "*MuJoCo Python Bindings Documentation*". (Per i dettagli tecnici sull'accesso alle strutture dati mjModel).
- ²²
Tan et al., "*Sim-to-Real: Learning Agile Locomotion For Quadruped Robots*", RSS 2018. (Sull'importanza della randomizzazione della dinamica per la locomozione).

Bibliografia

1. env.zip
2. Sim-to-Real Transfer of Deep Reinforcement Learning Agents for Online Coverage Path Planning - arXiv, accesso eseguito il giorno dicembre 12, 2025, <https://arxiv.org/html/2406.04920v2>
3. Sim-to-Real Transfer for Locomotion Tasks on Legged Robots: A Survey - HULKS, accesso eseguito il giorno dicembre 12, 2025, https://hulks.de/files/PA_Luis-Scheuch.pdf
4. Robot Learning From Randomized Simulations: A Review - Frontiers, accesso eseguito il giorno dicembre 12, 2025, <https://www.frontiersin.org/journals/robotics-and-ai/articles/10.3389/frobt.2022.799893/full>
5. Active Domain Randomization - arXiv, accesso eseguito il giorno dicembre 12, 2025, <https://arxiv.org/pdf/1904.04762>
6. Domain Randomization via Entropy Maximization - arXiv, accesso eseguito il giorno dicembre 12, 2025, <https://arxiv.org/html/2311.01885v2>
7. Sim2Real Transfer for Deep Reinforcement Learning with Stochastic State Transition Delays - Amazon Science, accesso eseguito il giorno dicembre 12, 2025, <https://assets.amazon.science/1f/8c/983e48114baaab0ae5f03e2fd4d5/sim2real-transfer-for-deep-reinforcement-learning-with-stochastic-state-transition-delays.pdf>

8. Sim2Real Transfer for Deep Reinforcement Learning with Stochastic State Transition Delays, accesso eseguito il giorno dicembre 12, 2025, <https://proceedings.mlr.press/v155/sandha21a/sandha21a.pdf>
9. Sim-to-real Transfer of Deep Reinforcement Learning Agents for Online Coverage Path Planning - arXiv, accesso eseguito il giorno dicembre 12, 2025, <https://arxiv.org/html/2406.04920v1>
10. Model Editing - MuJoCo Documentation, accesso eseguito il giorno dicembre 12, 2025, <https://mujoco.readthedocs.io/en/stable/programming/modeledit.html>
11. Is the joint damping setting working correctly in mujoco models? · Issue #1916 · bulletphysics/bullet3 - GitHub, accesso eseguito il giorno dicembre 12, 2025, <https://github.com/bulletphysics/bullet3/issues/1916>
12. Impact of Static Friction on Sim2Real in Robotic Reinforcement Learning - ResearchGate, accesso eseguito il giorno dicembre 12, 2025, https://www.researchgate.net/publication/389547180_Impact_of_Static_Friction_on_Sim2Real_in_Robotic_Reinforcement_Learning
13. Impact of Static Friction on Sim2Real in Robotic Reinforcement Learning - arXiv, accesso eseguito il giorno dicembre 12, 2025, <https://arxiv.org/html/2503.01255>
14. Python - MuJoCo Documentation, accesso eseguito il giorno dicembre 12, 2025, <https://mujoco.readthedocs.io/en/stable/python.html>
15. Modeling - MuJoCo Documentation, accesso eseguito il giorno dicembre 12, 2025, <https://mujoco.readthedocs.io/en/stable/modeling.html>
16. Add Automatic Domain Randomization (ADR) Framework for Robust Sim-to-Real Transfer · Issue #527 · google-deepmind/dm_control - GitHub, accesso eseguito il giorno dicembre 12, 2025, https://github.com/google-deepmind/dm_control/issues/527
17. Generating Automatic Curricula via Self-Supervised Active Domain Randomization - arXiv, accesso eseguito il giorno dicembre 12, 2025, <https://arxiv.org/abs/2002.07911>
18. bay3s/auto-dr: Automatic Domain Randomization (ADR) proposed in "Solving Rubik's Cube with a Robot Hand" - GitHub, accesso eseguito il giorno dicembre 12, 2025, <https://github.com/bay3s/auto-dr>
19. Flow-based Domain Randomization for Learning and Sequencing Robotic Skills - arXiv, accesso eseguito il giorno dicembre 12, 2025, <https://arxiv.org/html/2502.01800v1>
20. Self-Paced Domain Randomization - Fabian Damken, accesso eseguito il giorno dicembre 12, 2025, <https://fabian.damken.net/projects/spdr.pdf>
21. MuJoCo: How can i change friction properties of geom with default class? - Reddit, accesso eseguito il giorno dicembre 12, 2025, https://www.reddit.com/r/reinforcementlearning/comments/wa58ft/mujoco_how_can_i_change_friction_properties_of/
22. Sim-to-Real: Learning Agile Locomotion For Quadruped Robots, accesso eseguito il giorno dicembre 12, 2025, <https://www.roboticsproceedings.org/rss14/p10.pdf>

